app.py

Reference copy only — use the original source file in the project folder to run the website.

```
import os
import sqlite3
from datetime import datetime
from functools import wraps
from pathlib import Path
from werkzeug.utils import secure_filename
from werkzeug.security import check_password_hash, generate_password_hash
from flask import Flask, render_template, request, redirect, url_for, flash, session, abort

BASE_DIR = Path(__file__).resolve().parent
DB_PATH = BASE_DIR / "instance" / "clinic.db"
UPLOAD_DIR = BASE_DIR / "static" / "uploads"
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "webp"}
MAX_UPLOAD_MB = 5

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "change-this-secret-before-deployment")
app.config["MAX_CONTENT_LENGTH"] = MAX_UPLOAD_MB * 1024 * 1024
app.config["UPLOAD_FOLDER"] = str(UPLOAD_DIR)

# Replace these details with the client's approved clinic information.
CLINIC = {
    "name": os.environ.get("CLINIC_NAME", "SmileCraft Dental Clinic"),
    "phone": os.environ.get("CLINIC_PHONE", "8483835641"),
    "address": os.environ.get("CLINIC_ADDRESS", "Update with the clinic's verified street address"),
    "maps_url": os.environ.get("CLINIC_MAPS_URL", "https://maps.google.com/?q=dental+clinic"),
    "hours": [("Monday – Saturday", "10:00 AM – 2:00 PM", "5:00 PM – 10:00 PM"),
              ("Sunday", "Call to confirm", "Call to confirm")],
}
SERVICES = [
    ("Teeth whitening", "Brighten your smile with a dentist-guided whitening consultation.", "sparkles"),
    ("Emergency care", "Contact the clinic for urgent dental concerns and guidance.", "heart-pulse"),
    ("Online dentist booking", "Request an appointment online and receive confirmation from the clinic.",
 "calendar-check"),
    ("Dentures & bridges", "Discuss tooth-replacement options designed around your needs.", "smile"),
    ("X-ray", "Dental imaging when clinically indicated by the dentist.", "scan"),
    ("Check-ups", "Routine examinations to help monitor your oral health.", "clipboard-check"),
    ("Dental implants", "Explore implant suitability and restorative treatment options.", "circle-dot"),
    ("Extractions", "Assessment and tooth removal when clinically necessary.", "shield-check"),
    ("Teeth reshaping", "Ask about conservative cosmetic contouring options.", "wand-sparkles"),
    ("Teeth cleaning", "Professional cleaning and personalised oral-hygiene advice.", "droplets"),
    ("Fillings and sealants", "Treatment options to restore or protect teeth.", "layers"),
    ("Bonding", "Cosmetic bonding consultation for selected chips, gaps or shape concerns.", "gem"),
    ("Paediatrics", "Friendly dental visits and age-appropriate oral-health guidance for children.",
"baby"),
    ("Mouth guards", "Ask about protective mouth guards for sports or grinding.", "shield"),
    ("Oral surgery", "Assessment and referral or treatment planning for surgical needs.", "activity"),
    ("Root canals", "Evaluation and treatment planning for an infected or inflamed tooth.", "git-
branch"),
    ("Laser dentistry", "Ask the dentist whether laser-assisted treatment is appropriate.", "zap"),
    ("Veneers & crowns", "Discuss restorative and cosmetic options for your smile.", "crown"),
    ("Cosmetic procedures", "Personalised consultation for suitable cosmetic dental options.", "heart"),
]

def get_db():
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    with get_db() as db:
        db.execute("""CREATE TABLE IF NOT EXISTS appointments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL, phone TEXT NOT NULL, email TEXT,
            service TEXT NOT NULL, preferred_date TEXT, preferred_time TEXT,
            message TEXT, status TEXT NOT NULL DEFAULT 'New',
            created_at TEXT NOT NULL)""")
        db.execute("""CREATE TABLE IF NOT EXISTS gallery (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            filename TEXT NOT NULL, caption TEXT NOT NULL DEFAULT '',
            created_at TEXT NOT NULL)""")
        db.execute("""CREATE TABLE IF NOT EXISTS admins (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL)""")
        # Create a first admin only when credentials are supplied through environment variables.
        username = os.environ.get("ADMIN_USERNAME")
        password = os.environ.get("ADMIN_PASSWORD")
```

```
        if username and password:
            existing = db.execute("SELECT id FROM admins WHERE username=?", (username,)).fetchone()
            if not existing:
                db.execute("INSERT INTO admins (username, password_hash) VALUES (?, ?)",
                           (username, generate_password_hash(password)))

def admin_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if not session.get("admin_id"):
            flash("Please sign in to access the clinic manager.", "warning")
            return redirect(url_for("admin_login"))
        return fn(*args, **kwargs)
    return wrapper

def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS

@app.context_processor
def inject_globals():
    return {"clinic": CLINIC, "services": SERVICES, "current_year": datetime.now().year}

@app.route("/")
def index():
    with get_db() as db:
        gallery = db.execute("SELECT * FROM gallery ORDER BY id DESC LIMIT 6").fetchall()
    return render_template("index.html", gallery=gallery)

@app.route("/appointment", methods=["POST"])
def appointment():
    name = request.form.get("name", "").strip()
    phone = request.form.get("phone", "").strip()
    email = request.form.get("email", "").strip()
    service = request.form.get("service", "").strip()
    preferred_date = request.form.get("preferred_date", "").strip()
    preferred_time = request.form.get("preferred_time", "").strip()
    message = request.form.get("message", "").strip()
    valid_services = {s[0] for s in SERVICES}
    if not name or not phone or service not in valid_services:
        flash("Please provide your name, phone number and a listed service.", "error")
        return redirect(url_for("index") + "#book")
    with get_db() as db:
        db.execute("""INSERT INTO appointments
            (name, phone, email, service, preferred_date, preferred_time, message, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
            (name, phone, email, service, preferred_date, preferred_time, message,
             datetime.now().strftime("%Y-%m-%d %H:%M")))
    flash("Your appointment request has been sent. The clinic will contact you to confirm the slot.",
"success")
    return redirect(url_for("index") + "#book")

@app.route("/admin/login", methods=["GET", "POST"])
def admin_login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        with get_db() as db:
            admin = db.execute("SELECT * FROM admins WHERE username=?", (username,)).fetchone()
        if admin and check_password_hash(admin["password_hash"], password):
            session.clear()
            session["admin_id"] = admin["id"]
            session["admin_username"] = admin["username"]
            return redirect(url_for("admin_dashboard"))
        flash("Incorrect username or password. Check your deployment environment settings.", "error")
    return render_template("admin_login.html")

@app.route("/admin/logout")
def admin_logout():
    session.clear()
    flash("You have been signed out.", "success")
    return redirect(url_for("index"))

@app.route("/admin")
@admin_required
def admin_dashboard():
    status = request.args.get("status", "All")
```

```
    query = request.args.get("q", "").strip()
    sql = "SELECT * FROM appointments WHERE 1=1"
    args = []
    if status != "All":
        sql += " AND status=?"
        args.append(status)
    if query:
        sql += " AND (name LIKE ? OR phone LIKE ? OR service LIKE ?)"
        args.extend([f"%{query}%"] * 3)
    sql += " ORDER BY id DESC"
    with get_db() as db:
        appointments = db.execute(sql, args).fetchall()
        counts = {
            "all": db.execute("SELECT COUNT(*) FROM appointments").fetchone()[0],
            "new": db.execute("SELECT COUNT(*) FROM appointments WHERE status='New'").fetchone()[0],
            "confirmed": db.execute("SELECT COUNT(*) FROM appointments WHERE status='Confirmed'").
fetchone()[0],
            "completed": db.execute("SELECT COUNT(*) FROM appointments WHERE status='Completed'").
fetchone()[0],
        }
        gallery = db.execute("SELECT * FROM gallery ORDER BY id DESC").fetchall()
    return render_template("admin_dashboard.html", appointments=appointments, counts=counts,
                           gallery=gallery, selected_status=status, query=query)

@app.route("/admin/appointment/<int:appointment_id>/status", methods=["POST"])
@admin_required
def update_appointment(appointment_id):
    status = request.form.get("status", "")
    if status not in {"New", "Confirmed", "Completed", "Cancelled"}:
        abort(400)
    with get_db() as db:
        db.execute("UPDATE appointments SET status=? WHERE id=?", (status, appointment_id))
    flash("Appointment status updated.", "success")
    return redirect(url_for("admin_dashboard"))

@app.route("/admin/gallery/upload", methods=["POST"])
@admin_required
def upload_photo():
    file = request.files.get("photo")
    caption = request.form.get("caption", "").strip()[:120]
    if not file or not file.filename:
        flash("Choose a photo first.", "error")
        return redirect(url_for("admin_dashboard") + "#gallery-manager")
    if not allowed_file(file.filename):
        flash("Please upload JPG, JPEG, PNG or WEBP images only.", "error")
        return redirect(url_for("admin_dashboard") + "#gallery-manager")
    safe_name = secure_filename(file.filename)
    unique_name = f"{datetime.now().strftime('%Y%m%d%H%M%S%f')}_{safe_name}"
    file.save(UPLOAD_DIR / unique_name)
    with get_db() as db:
        db.execute("INSERT INTO gallery (filename, caption, created_at) VALUES (?, ?, ?)",
                   (unique_name, caption, datetime.now().strftime("%Y-%m-%d %H:%M")))
    flash("Photo uploaded to the public gallery.", "success")
    return redirect(url_for("admin_dashboard") + "#gallery-manager")

@app.route("/admin/gallery/<int:photo_id>/delete", methods=["POST"])
@admin_required
def delete_photo(photo_id):
    with get_db() as db:
        photo = db.execute("SELECT * FROM gallery WHERE id=?", (photo_id,)).fetchone()
        if photo:
            path = UPLOAD_DIR / photo["filename"]
            if path.exists():
                path.unlink()
            db.execute("DELETE FROM gallery WHERE id=?", (photo_id,))
    flash("Photo removed from the gallery.", "success")
    return redirect(url_for("admin_dashboard") + "#gallery-manager")

@app.errorhandler(413)
def too_large(_error):
    flash(f"Photo is too large. Maximum upload size is {MAX_UPLOAD_MB} MB.", "error")
    return redirect(url_for("admin_dashboard") + "#gallery-manager")

@app.route("/health")
def health():
    return {"status": "ok"}
```

```
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
init_db()

if __name__ == "__main__":
    app.run(debug=os.environ.get("FLASK_DEBUG") == "1", host="127.0.0.1", port=int(os.environ.get("PORT",
 5000)))
```